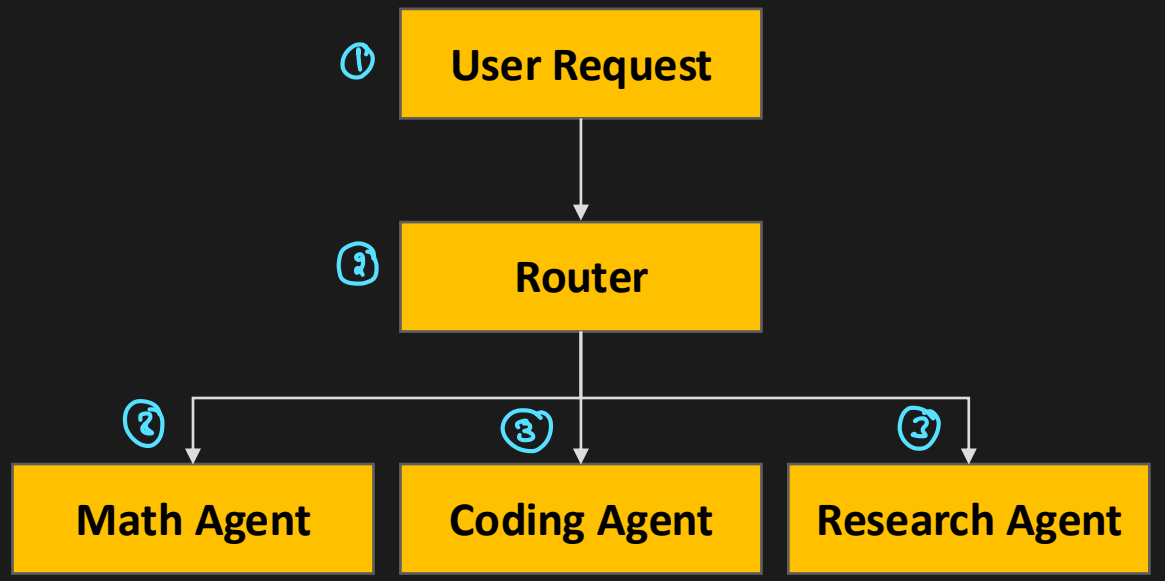
Routing

Introduction

- The **Routing Design Pattern** is an Agentic AI pattern in which an incoming task is first classified or analyzed by a router, and then the task is dynamically sent to the **most appropriate** agent, model, tool, or workflow
- Instead of using one agent to handle every type of request, routing allows a system to have **specialized agents** and intelligently decide which one should handle each request
- In a traditional application, routing usually means
  - URL → Controller → Service
- In Agentic AI, routing is more dynamic
- The decision can be based on:
  - User intent
  - Task type
  - Domain
  - Complexity
  - Required tools
  - User permissions



Why Do We Need Routing

- **The "One Model for Everything" Problem**
  - Without routing, a single do-everything prompt forces one model on every input
  - This creates a permanent tradeoff
    - Use an expensive frontier model for everything → you overpay on trivial queries (e.g., "capital of France?")
    - Use a cheap small model for everything → you underperform on hard tasks (e.g., complex multi-step proofs)
  - Routing **breaks this bind** by separating "what kind of question is this?" from "answer it," allowing each component to be optimized independently—including the choice of model, prompt, and tools
- **The Heterogeneity Assumption**
  - Extensive research validates that **no single agent or backbone uniformly dominates** across tasks
  - Performance rankings vary substantially across benchmarks and backbones, even under identical prompt templates
  - The best agent on one benchmark is often suboptimal on another
  - This heterogeneity is the empirical justification for routing: a principled, task-aware mechanism outperforms a one-size-fits-all approach

Router vs Agent

- A common misconception is: The router itself is an agent
- A router can be implemented using
  - Rule-based logic
  - Machine learning classifier
  - LLM-based routing
  - Embedding similarity
  - Hybrid routing

LLM-Based Routing



- One of the most common implementations in modern Agentic AI is an LLM acting as the router
- For example:
  - **User request**
    - "Analyze Apple's latest earnings and explain how they might affect the stock."
- The router might produce:

```
{
  "route": "financial_research",
  "confidence": 0.94
}
```

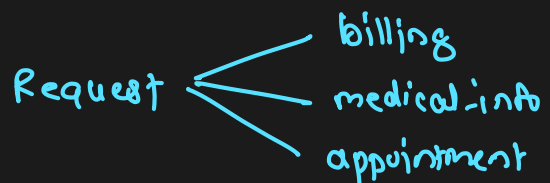
- The system then invokes: Financial Research Agent
- instead of the general-purpose agent

Routing Strategies



- Routing Based on Intent → LLM as a Router
- Routing Based on Domain
- Routing Based on Task Complexity
- Routing Based on Cost
- Routing Based on Latency
- Routing Based on Available Tools

Common Routing Topologies



- **Flat (Single-Level) Routing**
  - One router classifies into N categories, each with a dedicated handler
  - Simplest and most common
  - Works well when the number of categories is manageable (roughly 3 – 10)
  - **Example:** Customer service queries split into refunds, technical support, and general questions
- **Hierarchical Routing**
  - A **router of routers**: a top-level triage agent picks a domain, then a domain-level router picks the specialist
  - This tames large systems where a flat router would face too many destinations at once
  - Cost: more hops and more places a decision can go wrong
- **Auction-Based Routing**
  - Instead of a central router pushing work to an agent, candidate agents bid on a request, and the best-fit bid wins
  - Suits dynamic agent pools where capabilities change frequently
  - Bidding adds coordination overhead, rarely worth it for small, fixed teams
- **Context-Aware Routing (Framework-Specific)**
  - The router reads the request, classifies it, and returns a Handoff(target=specialist) directly from the classify tool
  - The framework resolves the routing target from the tool result, bypassing transition graph conditions entirely
  - This avoids a known stall issue where ContextEquals conditions only evaluate on substantive packets

When to Use Routing

- **Strong Fit**
  - **Inputs come in distinguishable types** that warrant materially different handling
  - **Classification can be handled accurately** by an LLM or traditional algorithm
  - **Complex tasks with distinct categories** are better handled separately
  - **Cost/quality optimization** matters—you want to spend expensive model calls only where they help
  - **Specialization is needed**—optimizing one input type shouldn't hurt performance on others
- **Weak Fit / Anti-Patterns**
  - **Truly uniform inputs** with no meaningful categories—routing adds overhead with no benefit
  - **Categories are not reliably distinguishable**—if the classifier can't pick accurately, routing errors compound
  - **Very small number of simple paths**—a simple conditional may suffice without a full routing layer